

Schroeder trees, monotonous and machine computations

P.Catoire, J.Fromentin

Univ. Littoral Côte d'Opale, UR 2597 LMPA, Laboratoire de Mathématiques Pures et Appliquées
Joseph Liouville F-62100 Calais, France .

April 3, 2025

Abstract

Keywords.

AMS classification.

Contents

1 Schroeder trees and chains in a poset

1.1 Partition of a set

Definition 1.1. Let $n \in \mathbb{N}$. A *partition* of $\llbracket 1, n \rrbracket$ is a subset $\{I_j\}_{j \in \llbracket 1, k \rrbracket}$ of $\llbracket 1, n \rrbracket$ composed of non-empty disjoint subsets such that $\llbracket 1, n \rrbracket = \bigcup_{j \in \llbracket 1, k \rrbracket} I_j$. We call each of its element I_j a *block*.

Definition 1.2. Let $S \subseteq U$ be a subset of $\mathbb{N}^*$, we call S to be *connected* with respect to U if and only if for all $s < t$ pair of elements of S , $U \cap \llbracket s, t \rrbracket \subseteq S$. Then, for each part $S \subseteq U$ can be written as a union of connected parts of U , called *connected components* of S with respect to U .

Any partition of a set can be represented as a diagram.

Example 1.1. For instance consider the partitions of the following set $I = \llbracket 1, 10 \rrbracket$:

- $\alpha_0 = \llbracket 1, 3 \rrbracket$, $\gamma_0 = \llbracket 4, 7 \rrbracket$ et $\beta_0 = \llbracket 7, 10 \rrbracket$.
- $\alpha_1 = \llbracket 1, 5 \rrbracket$, $\gamma_1 = \{6, 9, 10\}$ et $\beta_1 = \{7, 8\}$.
- $\alpha_2 = \llbracket 1, 5 \rrbracket$, $\gamma_2 = \{6, 8, 10\}$ et $\beta_2 = \{7, 9\}$.

Definition 1.3. A partition π of $\llbracket 1, n \rrbracket$ is called *non-crossing* if it fulfils the following property :

$$\forall (\gamma_1, \gamma_2) \in \pi^2, \left[((i, l) \in \gamma_1^2 \wedge (j, m) \in \gamma_2^2 \wedge i \leq j \leq l \leq m) \implies \gamma_1 = \gamma_2 \right].$$

*Contact: pierre.catoire@univ-littoral.fr

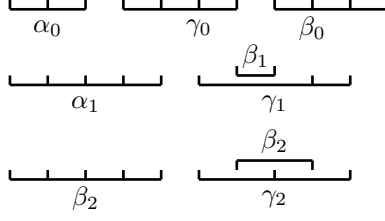


Figure 1: Some diagrams representing those partitions

Taking back the example ??, the first and second one are non-crossing partitions but not the last one.

Notation 1.1. We will denote $\mathcal{NC}_n$ the set of non-crossing partitions of $\llbracket 1, n \rrbracket$.

Definition 1.4. A partition π of $\llbracket 1, n \rrbracket$ is called *monotonous* if it is a non-crossing partition and owes a total order λ over its blocks π such that:

$$V \text{ is nested in } W \implies W <_\lambda V. \quad (1)$$

So, we can number those blocks $\pi = \{V_1, V_2, \dots, V_l\}$ as $V_1 \cup V_2 \cup \dots \cup V_l = \llbracket 1, n \rrbracket$ such that for all $i, j \in \llbracket 1, l \rrbracket$, we have :

$$(V_i <_\lambda V_j) \iff i < j.$$

Definition 1.5. Let $n \in \mathbb{N} \setminus \{0\}$. A partition π of $\llbracket 1, n \rrbracket$ is called *interval* if it only has blocks which are their own connected components defined at ??. We denote by $\mathcal{IP}_n$ the set of intervals partitions.

In the previous example (??), only the first one is an interval partition.

1.2 The poset of interval partitions

Notation 1.2. Let $(P \leq)$ be a poset. We denote $<$ the binary relation of P defined for any couple $(x, y) \in P^2$ by:

$$x < y \iff x \leq y \text{ and } x \neq y.$$

Definition 1.6. Let $n \in \mathbb{N} \setminus \{0\}$. We define an order over the set $\mathcal{IP}_n$, given for any $I, J \in \mathcal{IP}_n$ by:

$$J \leq I \iff \forall J_j \in I, \exists I_i \in J, J_i \subseteq I_j.$$

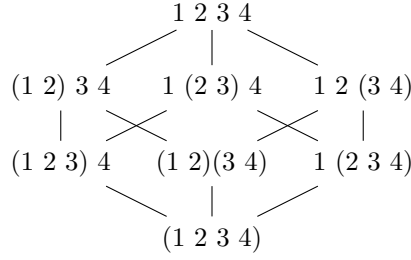
In other words, J is smaller than I if J is *thinner* than I . For any $(I, J) \in \mathcal{IP}_n$ such that $I \leq J$, we say it is a *covering relation* if $I \neq J$ and there does not exist $K \in \mathcal{IP}_n$ such that $I < K < J$.

Definition 1.7 (Chain of a poset). Let $(P, \leq)$ be a poset. A *chain* in P is a sequence of elements of P denoted $c = (c_1, \dots, c_k)$ where for all $i \in \llbracket 1, k-1 \rrbracket$, $c_i < c_{i+1}$. A chain $c = (c_1, \dots, c_k)$ is called *maximal*, if for every $i \in \llbracket 1, k-1 \rrbracket$, there does not exist any element $x \in P$ such that $c_i < x < c_{i+1}$. We call k the *length* of the chain c . A k -*chain* is a chain of length $k \in \mathbb{N}^*$.

Suppose that $(P, \leq)$ has a minimum and a maximum element respectively denoted $\min(P)$ and $\max(P)$. We call an *initial chain* every chain beginning with $\min(P)$ and ending with $\max(P)$. We denote $\mathcal{C}(P)$ the set of initial chains of the poset P .

We can represent a poset as a graph where the edges represent covering relations of the poset.

Figure 2: The $\mathcal{IP}_4$ poset



Example 1.2. For instance, let us draw the poset $\mathcal{IP}_4$ at figure ??:

Definition 1.8 (composition of chains in the poset $\mathcal{IP}$). Let $n, m \in \mathbb{N} \setminus \{0\}$ and consider c a k -chain of $\mathcal{IP}_n$ and d a l -chain of $\mathcal{IP}_m$. We define many compositions of the two chains c and d as the chain of $\mathcal{IP}_{n+m}$ defined by:

$$\begin{aligned} c \circ d &= (c_1 \cdot d_1, c_2 \cdot d_1, \dots, c_k \cdot d_1, c_k \cdot d_2, \dots, c_k \cdot d_l, \llbracket 1, n+m \rrbracket), \\ c \bullet d &= (c_1 \cdot d_1, c_1 \cdot d_2, \dots, c_1 \cdot d_l, c_2 \cdot d_l, \dots, c_k \cdot d_l, \llbracket 1, n+m \rrbracket), \\ c \star d &= (c_1 \cdot d_1, c_2 \cdot d_2, \dots, c_{\min(k,l)} \cdot d_{\min(k,l)}, \dots, c_k \cdot d_l), \end{aligned}$$

where $\cdot$ is the concatenation production of partitions shifting the second one by the maximum of the first one.

1.3 Schroeder trees and comb representations

Let us remind the following definition:

Definition 1.9. A *Schroeder tree* is a rooted tree such that any vertices which is not a leaf has at least 2 children. This set can be graded with the number of leaves minus 1. For any $n \in \mathbb{N} \setminus \{0\}$, we define $\mathcal{T}_n$ the set of Schroeder trees with $n+1$ leaves.

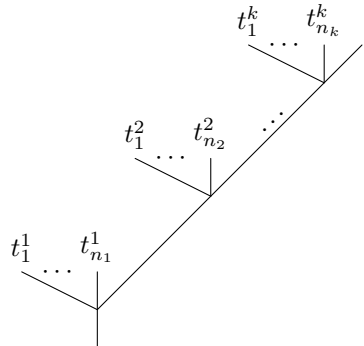
Notation 1.3. If $t \in \mathcal{T}_n$, we define $n := \deg(n)$. We will also denote c_n the corolla of degree n .

Remark 1.1. As every node of these trees which is not a leaf has at least 2 children, we will not draw the nodes of a tree any more.

Example 1.3. For instance, here is some examples of Schroeder trees:

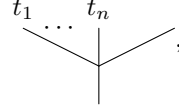


Let t be a tree. It t can always be seen as:



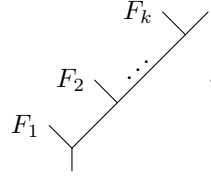
where k is the number of nodes on the right-most branch of t and for $i \in \llbracket 1, k \rrbracket$, $n_i + 1$ is the number of sons of the i -th node of this branch.

Notation 1.4. Let $F = t_1 \dots t_n$ be a forest composed of n trees. Instead of writing:



we will write: $F \begin{array}{c} \diagup \diagdown \\ | \end{array}$.

As a consequence, we deduce that all tree t can be seen as:



where $F_1, \dots, F_k$ are non-empty forests.

Definition 1.10. The writing defined above is the writing of t as a *right comb*. Proceeding the same way, looking at the left branch of the root of t , we get the writing of t as a *left comb*.

1.4 Our objective

Definition 1.11. Let $\mathbb{K}$ be a field and A be vector space of $\mathbb{K}$. We say $(A, \prec, \cdot, \succ)$ is a *tridendriform algebra* if $\prec, \cdot$ and $\succ$ are all linear maps from $A \otimes A$ to A such that for any $a, b, c \in A$:

$$(a \prec b) \prec c = a \prec (b * c), \quad (2)$$

$$(a \succ b) \prec c = a \succ (b \prec c), \quad (3)$$

$$(a * b) \succ c = a \succ (b \succ c), \quad (4)$$

$$(a \succ b) \cdot c = a \succ (b \cdot c), \quad (5)$$

$$(a \prec b) \cdot c = a \cdot (b \succ c), \quad (6)$$

$$(a \cdot b) \prec c = a \cdot (b \prec c), \quad (7)$$

$$(a \cdot b) \cdot c = a \cdot (b \cdot c), \quad (8)$$

where $*$ is the *associative product* of the tridendriform algebra defined for any $a, b \in A$ by $a * b := a \prec b + a \cdot b + a \succ b$. We call the products $\succ, \prec, \cdot$ respectively right, left and middle.

The free tridendriform algebra is built on the vector space $\mathbb{K} \mathcal{T} \oplus \mathbb{K} \cdot |$ where $|$ is the unit of the product $*$. We will denote it by $(\mathcal{A}, \prec, \cdot, \succ)$. We have a combinatorial interpretation of the product $*$. For this let us introduce:

Definition 1.12 (Quasi-shuffle). Let $k, l \in \mathbb{N} \setminus \{0\}$. A (k, l) -*quasi-shuffle* is a surjective map $\sigma : \llbracket 1, k + l \rrbracket \rightarrow \llbracket 1, n \rrbracket$ surjective for some positive integer n such that:

$$\sigma(1) < \dots < \sigma(k) \text{ and } \sigma(k+1) < \dots < \sigma(k+l).$$

We will denote $\text{QSh}(k, l)$ the set of all (k, l) -quasi-shuffles.

Let t, s two trees different from $|$. We look t as a right comb and s as a left comb. In other words, we put:

$$t = \begin{array}{c} F_k \\ \diagup \\ F_2 \dots \\ \diagup \\ F_1 \end{array} \quad \text{and} \quad s = \begin{array}{c} F_{k+l} \\ \diagdown \\ \dots \\ \diagdown \\ F_{k+2} \\ \diagdown \\ F_{k+1} \end{array},$$

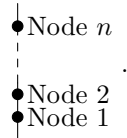
where for all $i \in \llbracket 1, k+l \rrbracket$, F_i is a non-empty forest. Here, k represents the number of nodes on the right-most branch of t and l is the number of nodes on the left-most branch of s .

Examples 1.1. For instance, we give some examples of trees and their decompositions as right and left combs:

- $\begin{array}{c} \diagup \diagdown \\ \diagup \diagdown \end{array} = F \begin{array}{c} Q \\ \diagup \diagdown \\ \diagup \diagdown \end{array} = \begin{array}{c} Q \\ \diagup \diagdown \\ \diagup \diagdown \end{array} F$ where $F = \begin{array}{c} \diagup \diagdown \end{array}$ and $Q = |$.
- $\begin{array}{c} \diagup \diagdown \\ \diagup \diagdown \end{array} = \begin{array}{c} F \\ \diagup \diagdown \\ \diagup \diagdown \end{array} = \begin{array}{c} \diagup \diagdown \\ \diagup \diagdown \end{array} F$ where $F = ||$.
- $\begin{array}{c} \diagup \diagdown \\ \diagup \diagdown \end{array} = H \begin{array}{c} \diagup \diagdown \\ \diagup \diagdown \end{array} = \begin{array}{c} Q \\ \diagup \diagdown \\ \diagup \diagdown \end{array} F$ where $H = \begin{array}{c} \diagup \diagdown \\ \diagup \diagdown \end{array}$, $Q = |$ and $F = \begin{array}{c} \diagup \diagdown \end{array}$.
- $\begin{array}{c} \diagup \diagdown \\ \diagup \diagdown \end{array} = F \begin{array}{c} G \\ \diagup \diagdown \\ \diagup \diagdown \end{array} = \begin{array}{c} J \\ \diagup \diagdown \\ \diagup \diagdown \end{array} I$ with $F = \begin{array}{c} \diagup \diagdown \end{array}$, $G = |$, $H = \begin{array}{c} \diagup \diagdown \end{array}$ and $I = \begin{array}{c} \diagup \diagdown \\ \diagup \diagdown \end{array}$, $J = ||$.

Definition 1.13. Let t and s be two trees which respective right comb representation and left comb representation are given above. We denote by k (respectively l) the number of nodes on the right-most (respectively left-most) branch of t (respectively s). Let σ be a (k, l) -quasi-shuffle which has for image $\llbracket 1, n \rrbracket$ for n a positive integer. We denote $\sigma(t, s)$ the tree obtained this way:

1. We first consider the ladder with n nodes:



2. For all $i \in \llbracket 1, k \rrbracket$, we graft F_i as the *left* son at the node $\sigma(i)$.
3. For all $i \in \llbracket k+1, k+l \rrbracket$, we graft F_i as *right* son at the node $\sigma(i)$.

Example 1.4. Consider the $(2, 2)$ -quasi-shuffle $\sigma = (1, 3, 2, 3)$. Let us take:

$$t = \begin{array}{c} F_2 \\ \diagup \diagdown \\ F_1 \end{array} \quad \text{and} \quad s = \begin{array}{c} F_4 \\ \diagup \diagdown \\ F_3 \end{array}. \quad \text{Then } \sigma(t, s) = \begin{array}{c} F_2 \quad F_4 \\ \diagup \quad \diagdown \\ F_1 \quad F_3 \end{array}.$$

Theorem 1.14. *Let t, s be two trees different from $|$ as described above. Then:*

$$t * s = \sum_{\sigma \in \text{QSh}(k, l)} \sigma(t, s).$$

Moreover:

$$\begin{aligned} t \prec s &= \sum_{\substack{\sigma \in \text{QSh}(k, l) \\ \sigma^{-1}(\{1\}) = \{1\}}} \sigma(t, s), \\ t \cdot s &= \sum_{\substack{\sigma \in \text{QSh}(k, l) \\ \sigma^{-1}(\{1\}) = \{1, k+1\}}} \sigma(t, s), \\ t \succ s &= \sum_{\substack{\sigma \in \text{QSh}(k, l) \\ \sigma^{-1}(\{1\}) = \{k+1\}}} \sigma(t, s). \end{aligned}$$

Surprisingly, $(\mathcal{A}, \prec, \cdot, \succ)$ can be endowed with a codendriform coproduct given by:

Theorem 1.15. *We define the map over $\mathcal{A}$ for all trees t by the formula:*

$$\begin{aligned} \Delta(t) &= \sum_{c \text{ admissible cut of } t} G^c(t) \otimes R^c(t), \\ \Delta(|) &= | \otimes |, \end{aligned}$$

where $R^c(t)$ is the component of t containing its root and $G^c(t) = G_1^c(t) * \dots * G_k^c(t)$, where $G_i^c(t)$ are naturally ordered from left to right. More over one can define:

$$\begin{aligned} \Delta_{\leftarrow}(t) &= \sum_{\substack{c \text{ admissible cut of } t \\ \text{the right-most leaf of } t \text{ is cut}}} G^c(t) \otimes R^c(t), \\ \Delta_{\rightarrow}(t) &= \sum_{\substack{c \text{ admissible cut of } t \\ \text{the right-most leaf of } t \text{ is not cut}}} G^c(t) \otimes R^c(t). \end{aligned}$$

Hence, $(\mathcal{A}, \prec, \cdot, \succ, \Delta_{\leftarrow}, \Delta_{\rightarrow})$ is a $(3, 2)$ -dendriform bialgebra.

Hence, a natural question is to compute the primitives of $\mathcal{A}$. Let us introduce the notations to reach the theorems for this purpose:

Definition 1.16. Let $(C, \Delta_{\leftarrow}, \Delta_{\rightarrow})$ be a dendriform coalgebra. We define:

$$\begin{aligned} \text{Prim}_{\leftarrow}(C) &:= \{c \in C \mid \tilde{\Delta}_{\leftarrow}(c) = 0\}, \\ \text{Prim}_{\rightarrow}(C) &:= \{c \in C \mid \tilde{\Delta}_{\rightarrow}(c) = 0\}, \\ \text{Prim}_{\text{Coass}}(C) &:= \{c \in C \mid \tilde{\Delta}(c) = 0\}, \\ \text{Prim}_{\text{Codend}}(C) &:= \{c \in C \mid \tilde{\Delta}_{\leftarrow}(c) = \tilde{\Delta}_{\rightarrow}(c) = 0\}. \end{aligned}$$

Theorem 1.17. *For all $n \in \mathbb{N}$, we define:*

$$\theta_n : \begin{cases} \text{Prim}_{\text{Coass}}(\mathcal{A})_n & \rightarrow & \text{Prim}_{\text{Codend}}(\mathcal{A})_{n+1} \\ a \otimes \bigvee & \mapsto & a \cdot \bigvee. \end{cases}$$

Then, for all $n \in \mathbb{N}$, θ_n is an isomorphism of vector spaces.

From the article [?] of M.Ronco, we recall the global construction:

Definition 1.18 (brace algebra). A *brace algebra* is a $\mathbb{K}$ -vector space B endowed for any $n \in \mathbb{N}$ of linear maps:

$$M_n : \begin{cases} B^{\otimes n} & \rightarrow B, \\ b_1 \otimes \cdots \otimes b_n & \mapsto M_n(b_1 \otimes \cdots \otimes b_n), \end{cases}$$

with $M_1 = \text{Id}_V$ such that for all $m, n \in \mathbb{N} \setminus \{0\}$, $a_1, \dots, a_m, b_1, \dots, b_n, c \in B$:

$$\begin{aligned} & M_{m+1}(a_1 \otimes \cdots \otimes a_m \otimes M_{n+1}(b_1 \otimes \cdots \otimes b_n \otimes c)) \\ &= \sum_{\mathbf{A}} M(A_0 \otimes M(A_1 \otimes b_1) \otimes A_2 \otimes M(A_3 \otimes b_2) \otimes A_4 \otimes \cdots \otimes A_{2n-2} \otimes M(A_{2n-1} \otimes b_n) \otimes A_{2n} \otimes c), \end{aligned}$$

where $\mathbf{A}$ is a of the tensor $a_1 \otimes \cdots \otimes a_m$ i.e $\mathbf{A} = (A_0, \dots, A_{2n}) \in T(B)^{2n}$ such that:

$$a_1 \otimes \cdots \otimes a_m = A_0 \otimes A_1 \otimes \cdots \otimes A_{2n-1} \otimes A_{2n}.$$

Example 1.5. One example of such a relation is:

$$M_2(a, M_2(b, c)) = M_3(a, b, c) + M_3(b, a, c) + M_2(M_2(a, b), c).$$

Notation 1.5. We will denote the free brace algebra generated by a set X by $\mathcal{B}r(X)$.

Definition 1.19. We define the following operators in $\mathcal{A}$:

$$\omega_{\prec} : \begin{cases} T(D)^+ & \rightarrow D, \\ x_1 \otimes \cdots \otimes x_n & \mapsto \begin{cases} x_1 & \text{if } n = 1, \\ \omega_{\prec}(x_1 \otimes \cdots \otimes x_{n-1}) \prec x_n & \text{else.} \end{cases} \end{cases}$$

For all $n \in \mathbb{N}$, we introduce the map $\omega : \bigcup_{n \in \mathbb{N}^*} D^n \rightarrow D$ defined for all $x_1, \dots, x_n \in D, y \in D$ by:

$$\begin{aligned} \omega(y) &= y, \\ \omega(x_1 \otimes \cdots \otimes x_n \otimes y) &:= \sum_{i=0}^n (-1)^{n-i} \omega_{\prec}(x_1 \otimes \cdots \otimes x_i) \succ y \prec \omega_{\succ}(x_{i+1}, \dots, x_n). \end{aligned}$$

Remark 1.2. This quantity is not ambiguous thanks to relation (??).

Notation 1.6. For all $n \in \mathbb{N}$, we denote by $\omega_{n-1} = \omega|_{D^{\otimes n}}$.

Here is one the second fundamental result:

Theorem 1.20. For all $n \in \mathbb{N}$:

$$\Omega_n : \begin{cases} T(\text{Prim}_{\text{Codend}}(\mathcal{A}))_n & \twoheadrightarrow \text{Prim}_{\text{Coass}}(\mathcal{A})_n, \\ x_1 \otimes \cdots \otimes x_k & \mapsto \omega(x_1 \otimes \cdots \otimes x_k), \end{cases} \quad (9)$$

This raise to a map Ω defined over $\mathcal{A}$ such that $\Omega|_{T(\text{Prim}_{\text{Codend}}(\mathcal{A}))_n} = \Omega_n$.

As a consequence, we have a recipee to produce the primitives of this algebra by induction:

This diagram allows us to compute $\text{Prim}_{\text{Coass}}(\mathcal{A})$ by induction over the degree of its elements. We will now look at the practical aspects to implement an algorithm on a computer.

1.5.2 From initial chains to trees

Notation 1.7. Let t be a tree with labelled leaves in a subset $I \subseteq \mathbb{N} \setminus \{0\}$, we put:

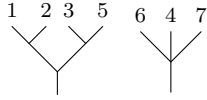
$$\text{Leaf}(t) = I.$$

Definition 1.22. We introduce $T(\mathcal{T})$ the tensor algebra with trees as letters. A *forest* is a word from $T(\mathcal{T})$.

Definition 1.23. Let $n \in \mathbb{N} \setminus \{0\}$. Let F be a forest of trees with n leaves indexed by $\llbracket 1, n \rrbracket$. We map a partition π of $\llbracket 1, n \rrbracket$ from $F = t_1 \dots t_k$ as:

$$\pi = \bigcup_{i=1}^k \{\text{Leaf}(t_i)\}.$$

We denote $\pi := \text{Par}(F)$.

Example 1.7. Consider the forest $F =$ . Then:

$$\text{Par}(F) = \{\{1, 2, 3, 5\}, \{4, 6, 7\}\}.$$

Remark 1.3. If we label the leaves of a forest F from left to right, we have $\text{Par}(F) \in \mathcal{IP}_n$.

Definition 1.24. Let n and k be two elements of $\mathbb{N} \setminus \{0\}$. Consider $\pi = \{\pi_1, \dots, \pi_s\}$ be an element of $\mathcal{IP}_n$ and let $F = t_1 \dots t_k$ be a forest of k trees with a total of n leaves indexed by $\llbracket 1, n \rrbracket$ from left to right such that:

$$\pi \leq \text{Par}(F).$$

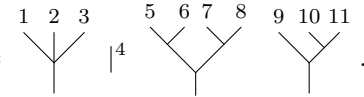
We define $\Psi_\pi(F)$ the forest defined by:

$$t_1 \vee \dots \vee t_{l_1} \quad t_{l_1+1} \vee \dots \vee t_{l_2+1} \quad \dots \quad t_{l_s} \vee \dots \vee t_k$$

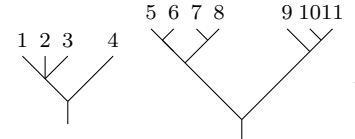
where $1 \leq l_1 \leq \dots \leq l_s \leq n$ and:

$$\forall s \in \llbracket 1, s \rrbracket, \bigcup_{i=1}^s \text{Leaf}(t_i) = \pi_i.$$

Example 1.8. Consider:

$$F =$$
 .

and $\pi = \{\llbracket 1, 4 \rrbracket, \llbracket 5, 11 \rrbracket\}$ then:

$$\Psi_\pi(F) =$$
 .

whereas the number of initial chains in the modified poset of figure ?? is:

$$1 + 6 + 4 = 11$$

the 4-th Schroeder number.

For more details let us consider the $\mathcal{IP}_5$ poset in figure ??:

After applying the rule on the first raw, one gets the figure ??.

This property of simplification in a poset needs more than just a poset. We need a map $\pi : P \times P \rightarrow \{0, 1\}$ that tells us whether we can perform a "parallelisation simplification rule", for instance consider the subposet of the poset $\mathcal{IP}_6$ given in figure ??:

3 Words and programming our problem

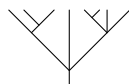
3.1 How to code a tree ?

A computer has no natural way to see what is a tree.

One way to see a Shroeder tree with n leaves is to consider them as a initial chain in the poset $\mathcal{IP}_n$ using the decomposition given at example ??. This decomposition is made such that one should make the merging of all the left blocks before merging any block on the left side.

Remark 3.1. In other words, we are reading the tree from left to right.

Example 3.1. The decomposition of the tree is:



$$1\ 2\ 3\ 4\ 5\ 6\ 7\ 8, (1\ 2\ 3)\ 4\ 5\ 6\ 7\ 8, (1\ 2\ 3)\ 4\ 5\ 6\ 7\ 8, (1\ 2\ 3)\ 4\ (5\ 6)\ 7\ 8, (1\ 2\ 3)\ 4\ (5\ 6\ 7\ 8), (1\ 2\ 3\ 4\ 5\ 6\ 7\ 8).$$

Any such code of length n can be interpreted, as a sequence of 0, 1 of length $n - 1$ with the bijection:

$$\varphi : \begin{cases} \mathcal{IP}_n & \rightarrow \{0, 1\}^{n-1} \\ P & \mapsto (\varphi(P)_i)_{i \in \llbracket 1, n-1 \rrbracket} \end{cases}$$

such that for any $i \in \llbracket 1, n - 1 \rrbracket$:

$$\varphi(P)_i = \begin{cases} 1 & \text{if } i \text{ and } i + 1 \text{ are un the same block in } P, \\ 0 & \text{else.} \end{cases}$$

Notation 3.1. For any $n \in \mathbb{N}$, $P \in \mathcal{IP}_n$, we will denote p the image of P by φ .

Let us introduce the *Hamming weight* for binary words:

Definition 3.1 (Hamming weight). Let $W = W_1 \dots W_k$ be a finite word over the alphabet $\{0, 1\}$, we define its *Hamming weight* as:

$$\omega(W) = |\{i \in \llbracket 1, k \rrbracket \mid W_i = 1\}|.$$

Example 3.2. For instance:

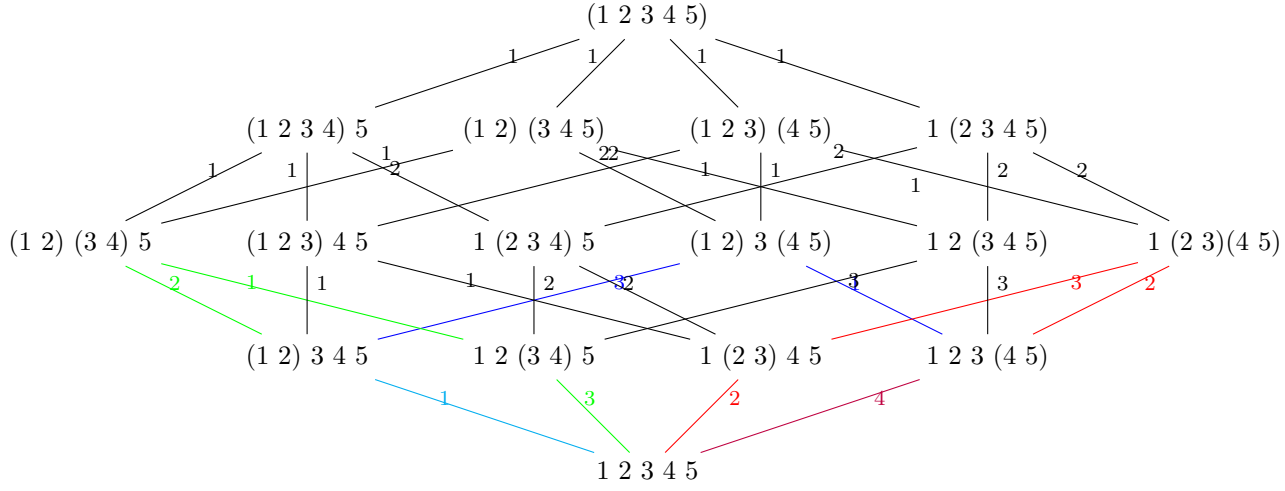
$$\omega(0011011) = 4.$$

the word 0011011 encodes the partition $1\ 2\ (3\ 4\ 5)\ (6\ 7\ 8) \in \mathcal{IP}_8$.

Now, we can identify any initial chain, as a sequence of words of $\{0, 1\}$.

Figure 8: The construction on $\mathcal{IP}_5$

(a) The 5-tree poset



(b) Simplifying the first row is enough

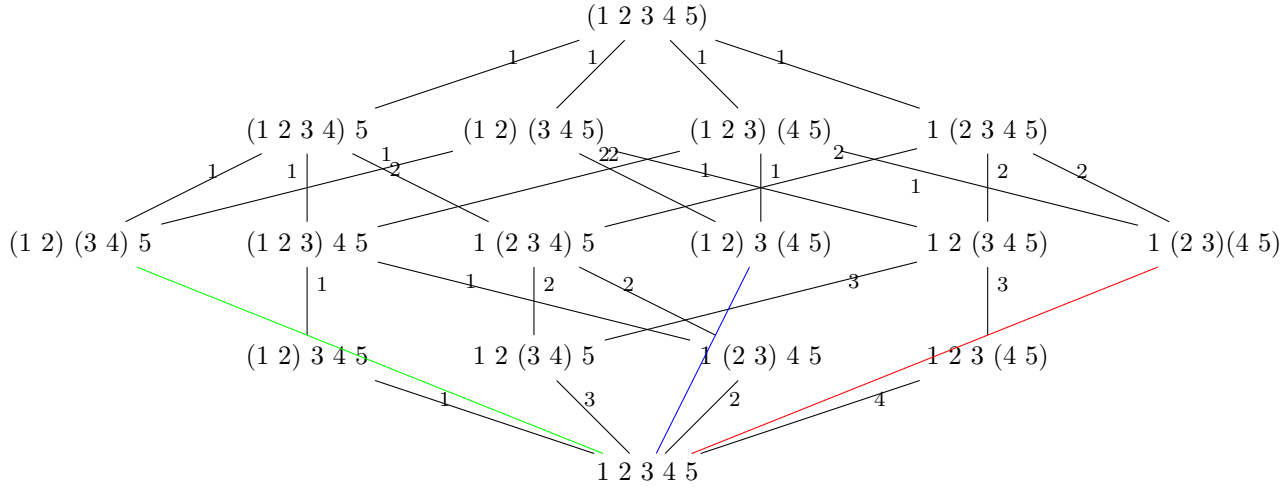
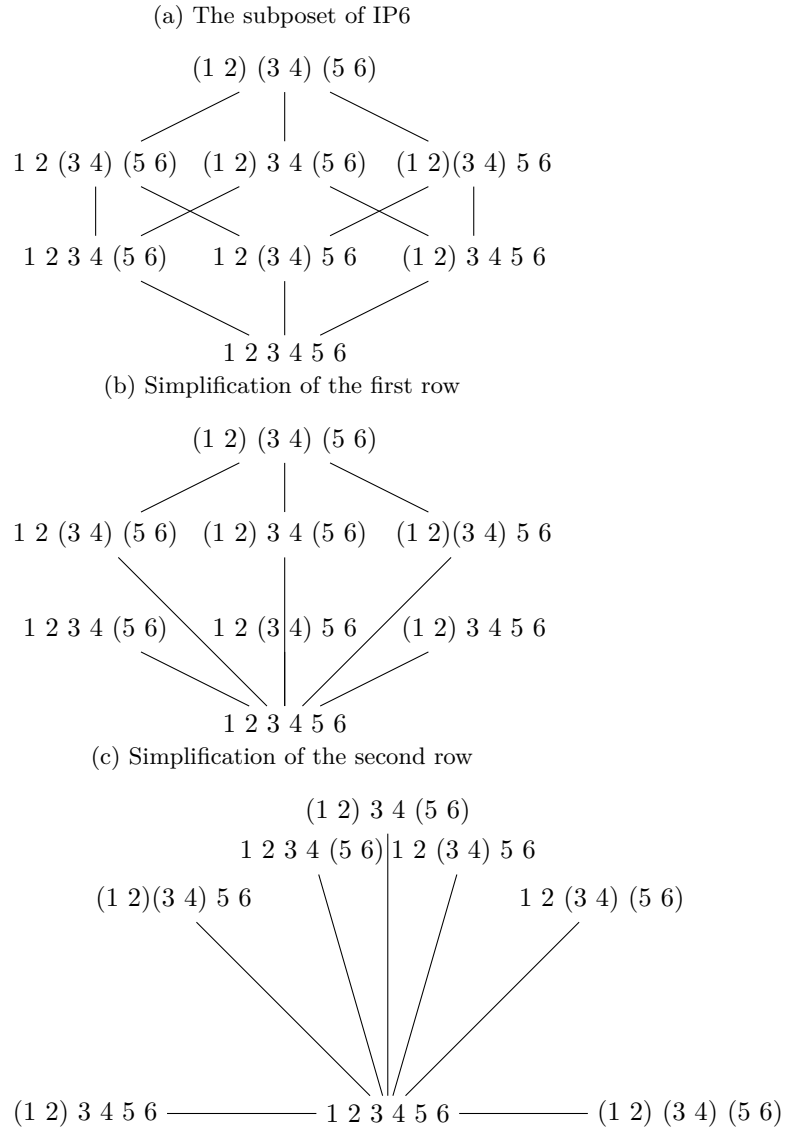
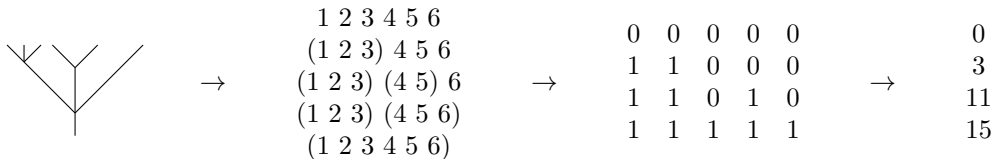


Figure 9: Example over IP6



Example 3.3. For instance:



In the following, we will delete the beginning and the ending of such chains. Moreover, reading the chains as a sequence of words over $\{0, 1\}$ one has:

Proposition 3.2. *Consider $(P_0, P_1, \dots, P_{k-1}, P_k)$ an initial chain in the poset $\mathcal{IP}_n$. Hence, $(p_0, p_1, \dots, p_{k-1}, p_k)$ is an increasing sequence for the Hamming weight and for any $i \in \llbracket 1, k-1 \rrbracket$:*

$$\begin{cases} \omega(p_0) &= 0, \\ \omega(p_k) &= n-1, \\ p_i \wedge p_{i+1} &= p_i, \\ p_i \vee p_{i+1} &= p_{i+1}. \end{cases}$$

Remark 3.2. We know each Schroeder tree t with n leaves can be seen as an initial chain in the poset $\mathcal{IP}_n$. Denoting this chain by $(P_0, \dots, P_k)$, we will call $(p_0, \dots, p_k)$ the *code* of the tree t . In example

We then adapt the definition ?? to chains of proposition ??:

Definition 3.3. Let $n, m \in \mathbb{N} \setminus \{0\}$ and consider S an increasing sequence for Hamming weight of length n and D another such sequence of length m . We define the following associative law:

$$S \circ D = (S_1 \cdot D_1, S_2 \cdot D_1, \dots, S_k \cdot D_1, S_k \cdot D_2, \dots, S_k \cdot D_l).$$

Remark 3.3. Note that the sequences defined in proposition ?? is closed under $\circ$.

Now, we need to make a bridge between the description of the operations $\prec, \cdot$ and $\succ$ from the point of view of trees to the point of view of chains defined in proposition ??.

3.2 How to describe the $\prec, \succ$ and $\cdot$ operations ?

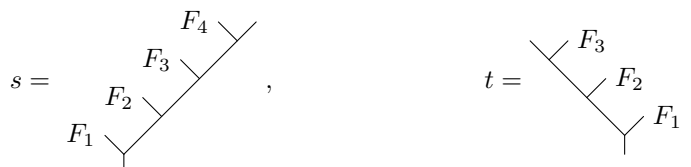
In this subsection, for sake of simplicity we will not consider shuffles (respectively quasi-shuffles) but element of a symmetric group (respectively maps a packed word with at most two repetitions of one letter). Note that definition ?? extends naturally to such objects.

3.2.1 Some clues on one example

For instance, consider the following element of S_7 :

$$\sigma = (1\ 2\ 5\ 3\ 6\ 7\ 4)$$

and consider two trees having the respective comb representations:



Hence:

$$\sigma(t, s) = \begin{array}{c} F_6 \\ F_5 \\ F_3 \\ F_7 \\ F_4 \\ F_2 \\ F_1 \end{array}$$

Can we interpret this action with the words over $\{0, 1\}$ encoding the tree ?
Take a simpler example:

$$t = \begin{array}{c} F_2 \\ F_1 \end{array} \text{ and } s = \begin{array}{c} F_3 \end{array}.$$

with $F_1 = \begin{array}{c} \diagup \\ \diagdown \end{array}$, $F_2 = \begin{array}{c} \diagup \\ \diagdown \end{array}$ and $F_3 = ||$. Consider $\sigma = (1 \ 3 \ 2)$, then:

$$\sigma(t, s) = \begin{array}{c} F_2 \\ F_1 \end{array} F_3 = \begin{array}{c} \diagup \\ \diagdown \end{array} \begin{array}{c} \diagup \\ \diagdown \end{array} \begin{array}{c} \diagup \\ \diagdown \end{array}.$$

With this simpler example, we notice t has 6 leaves and s has 3 leaves. So $\sigma(t, s)$ has 8 leaves.
Now, let us look at the code of these two trees:

$$t = \begin{array}{cccccc} 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 \end{array} \text{ et } s = \begin{array}{cc} 0 & 0 \\ 1 & 1 \end{array}$$

The "code" of $\sigma(t, s)$ is:

$$\begin{array}{cccccccc} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ F_1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ F_2 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ ; F_2 \cdot g & 1 & 0 & 1 & 1 & 1 & 0 & 0. \\ F_3 & 1 & 0 & 1 & 1 & 1 & 1 & 1 \\ F_3 \cdot g^2 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ g \cdot F_1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{array} \quad (10)$$

In equation ??, this is not the real code because one should delete the line indexed with F_3 . But it shows how to build from a quasi-shuffle the code of this tree with the labels. Actually, when one sees a F_i on the left hand side of the table in equation ??, one should copy the code of F_i at the indices of the leaves in F_i in the shuffled tree, and when one sees a $F_i \cdot g^j$ where j is the number of forests in F_i (respectively $g^j \cdot F_i$) it replaces j zeros reading from right to left (respectively from left to right) beginning with the rightmost (respectively leftmost) zero touching the code of F_i .

More rigorously, let us define:

Definition 3.4 (forest code). Let F be a forest $F = t_1 \cdots t_k$. For any $i \in \llbracket 1, k \rrbracket$, put C_i the code of the tree t_i . The *forest code* of F is:

$$C_1 \circ 0 \circ C_2 \circ 0 \circ \cdots \circ 0 \circ C_k.$$

Notation 3.2. We put $C_{|}$ the code given by 0.

Example 3.4. Let $F = \bigvee \bigvee |$, its code C is:

$$\begin{array}{ccccc} 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \end{array}$$

Remark 3.4. The code of $|$ is restrained to 0, hence the result of $C \circ C_{|}$ is simply the concatenation of a zero to every word in the code C .

Notation 3.3. Denote C_i the code of any forest F_i for $i \in \llbracket 1, n \rrbracket$.

We introduce a family of operators on the set of forests codes:

Definition 3.5 (Left and right closure operator). Let S be a sequence of 0 and 1 with j zeros. Let us denote:

$$S = 0^{n_0} 1^{m_1} 0^{n_1} \dots 0^{n_{l-1}} 1^{m_l} 0^{n_l}.$$

Let $k \in \llbracket 1, j \rrbracket$. We define the *right* (respectively *left*) *closure operator of order k* over the set of code with rightmost (respectively leftmost) element equal to 0. We denote it by $\leftarrow g^k$ (respectively $g^k \rightarrow$). The image of the sequence S is the same sequence where we have replaced the k firsts zeros by ones reading from right to left (respectively from left to right).

Examples 3.1. For instance:

$$\begin{aligned} 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 0 &\leftarrow g^3 = 0 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1 \ 1, \\ g^3 \rightarrow 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 0 &= 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0. \end{aligned}$$

Notation 3.4. We denote $\mathcal{C}$ the set of forests code. Moreover, we define:

1. $\mathcal{C}_0$ the subset of $\mathcal{C}$ such that the forests code ends with 0;
2. ${}_0\mathcal{C}$ the subset of $\mathcal{C}$ such that the forests code begins with 0;

Definition 3.6. Let C be a forest code. We define two following maps:

$$L_0 : \begin{cases} \mathcal{C} &\rightarrow \mathcal{C}_0 \\ C &\mapsto C \circ C_{|}, \end{cases} \quad {}_0L : \begin{cases} \mathcal{C} &\rightarrow {}_0\mathcal{C} \\ C &\mapsto C_{|} \circ C. \end{cases}$$

Remark 3.5. For trees, this operation is just the right or left concatenation of $|$.

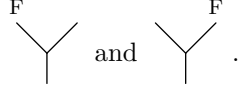
Let us introduce:

Definition 3.7 (The forest code grafting operator). Let C and D be two forest code and let n and m be the number of zeros in C and D . Put F and G the forests associated respectively to C and D . We define the maps $f_r : \mathcal{C}_0 \times \mathcal{C} \rightarrow \mathcal{C}$ and $f_l : {}_0\mathcal{C} \times \mathcal{C} \rightarrow \mathcal{C}$ defined by:

$$\begin{aligned} f_r(D, C) &= (C \circ D) \leftarrow g^{n+m}, \\ f_l(D, C) &= g^{n+m} \rightarrow (D \circ C). \end{aligned}$$

The trees corresponding to those codes are:

Example 3.5. Consider F a forest and its forest code C . Then, $f_r(C|, C)$ and $f_l(C|, C)$ are respectively encoding the following trees:



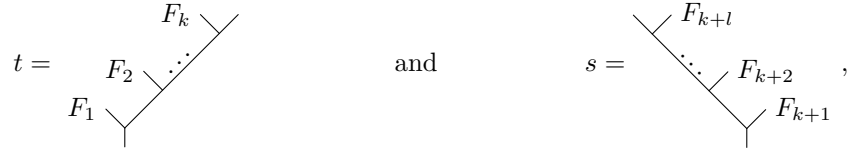
Lemma 3.8. Let F and G be two forests with respective code C_F and C_G . Then, the codes below correspond to the trees:

$$f_l(L_0(C_G), C_F) \rightarrow \begin{array}{c} G \quad F \\ \diagdown \quad \diagup \\ \text{---} \end{array}, \quad f_r({}_0L(C_G), C_F) \rightarrow \begin{array}{c} F \quad G \\ \diagup \quad \diagdown \\ \text{---} \end{array}.$$

Proof. Easy. □

Remark 3.6. The number of zeros in a forest code is exactly the number of trees in the forest associated to it.

Consider two trees with the following combs representations:



Let us denote for any $i \in \llbracket 1, k+l \rrbracket$, C_i the forest code of F_i and n_i the number of forests of C_i . Put C_t the code of t and C_s the code of s . Hence, their respective code are (with polish notations):

$$C_t = f_r \underbrace{L_0 f_r \dots L_0 f_r}_{(k-1) \text{ times}} C| C_k \dots C_2 C_1,$$

$$C_s = f_l \underbrace{{}_0L f_l \dots {}_0L f_l}_{(l-1) \text{ times}} C| C_k \dots C_2 C_1.$$

We introduce a final notation before giving the action of quasi-shuffles over those tree codes:

Notation 3.5. Let C be a code, we denote $\text{len}(C)$ the length of C .

Definition 3.9. Let $\sigma \in S_{k+l}$ and put C_i the code of the forest F_i for all $i \in \llbracket 1, k+l \rrbracket$ and m_i the number length of forests of C_i . Put $n = \max(\sigma)$.

1. put $\tau_1 = \sigma^{-1}|_{\llbracket 1, k \rrbracket}$ and begin with:

$$D_0 = L_0(C_{\tau_1(1)}) \circ \dots \circ L_0(C_{\tau_1(k)}).$$

2. put $\tau_2 = \sigma^{-1}|_{\llbracket k+1, k+l \rrbracket}$ and for all $i \in \llbracket 1, n \rrbracket$ compute iteratively:

$$D_i = \begin{cases} D_{i-1} \circ C_{j'} \leftarrow g^{m_j + m_{j'} + 1} & \text{if } \exists (j, j') \in \llbracket 1, k \rrbracket \times \llbracket k+1, k+l \rrbracket, \sigma(j') = \sigma(j) = i, \\ D_{i-1} \leftarrow g^{m_j + 1} & \text{if } \exists j \in \llbracket 1, k \rrbracket, \sigma(j) = i \text{ and } i \notin \text{Im}(\tau_2), \\ D_{i-1} \circ C_{j'} \leftarrow g^{m_{j'} + 1} & \text{if } \exists j' \in \llbracket k+1, k+l \rrbracket, \sigma(j') = i \text{ and } i \notin \text{Im}(\tau_1). \end{cases}$$

Define $D_n := \sigma(C_t, C_s)$.